ROBUST REINFORCEMENT LEARNING UNDER ENVIRONMENTAL UNCERTAINTY AND ADVERSARIAL DISTURBANCES

MID-SEMESTER PROJECT REPORT

Ayush Chauhan
2023B5A71089P
Department of Computer Science

ABSTRACT

Deep reinforcement learning (RL) policies are typically trained and evaluated within identical, stationary simulation environments assuming noise-free observations, instantaneous actuation, and fixed physical parameters. When deployed in the real world, however, agents encounter sensor noise, adversarial spoofing, actuation latency, and shifted physical dynamics. In this mid-semester study, we investigate the baseline fragility of standard continuous-control RL policies across two MuJoCo locomotion benchmarks with contrasting stability mechanics: `Walker2d-v5` (an upright bipedal walker vulnerable to falling) and `HalfCheetah-v5` (a mechanically stable planar runner). We train Soft Actor-Critic (SAC) agents for $300{,}000$ steps across multiple random seeds on clean environments, freeze the learned policies, and systematically stress-test them across five graded disturbance types spanning three channels: sensor inputs (random Gaussian noise and 10-step white-box Projected Gradient Descent adversarial attacks), actuation (control-step action delay), and physical dynamics (body mass scaling and ground friction scaling). For each disturbance, we quantify the degradation onset (ϵ_{80}) and failure breaking point (ϵ_{50}). Our empirical findings reveal that (1) action latency is catastrophic, with a single delayed step reducing return by 31% on `Walker2d-v5` and 70% on `HalfCheetah-v5`, and two steps causing total policy collapse; (2) white-box adversarial sensor perturbations are $2.4\times$ to $3.2\times$ more damaging than random Gaussian noise of identical magnitude, inducing twice the policy action deviation; and (3) sensitivity to physical parameter shifts is strongly asymmetric and dictated by robot mechanics. These failure curves establish the diagnostic foundation for our end-semester investigation into cross-perturbation robustness transfer.

1 Introduction and Motivation

Reinforcement learning (RL) formulates sequential decision-making under uncertainty by optimizing a policy π within a Markov Decision Process (MDP) $\mathcal{M}_{\text{train}} = (\mathcal{S}, \mathcal{A}, P_{\text{train}}, R, \gamma)$. In standard empirical practice, the evaluation environment $\mathcal{M}_{\text{test}}$ is identical to $\mathcal{M}_{\text{train}}$. In practical robotic deployment, this assumption is routinely violated: onboard sensors experience thermal noise or malicious spoofing, communication buses introduce actuation delays, payloads alter the robot's mass and inertia, and surface conditions alter contact friction (Gu et al., 2025).

The overarching research question of this project is:

> *Can an RL agent continue to perform well when the deployment environment differs from training conditions—including under adversarial disturbances—and which type of distribution shift is most damaging to a trained policy?*

Before designing or evaluating robust training strategies, it is essential to rigorously characterize **how, where, and at what intensity standard non-robust RL policies fail**. In this mid-semester phase, we address this diagnostic question through four concrete contributions:

1. We train baseline Soft Actor-Critic (SAC) (Haarnoja et al., 2018) agents to strong performance on two continuous-control MuJoCo environments with identical state-action dimensions (17-D state, 6-D action) but contrasting failure mechanics: `Walker2d-v5` and `HalfCheetah-v5`.

2. We implement a modular perturbation suite injecting five graded disturbances across three MDP interaction channels: random sensor noise, white-box adversarial sensor attacks, actuation delay, body mass scaling, and ground friction scaling.

3. Using Common Random Numbers (CRN) across $2{,}580$ evaluation episodes, we map normalized degradation curves and compute exact interpolation thresholds for degradation onset (ϵ_{80}, 20% loss) and policy failure (ϵ_{50}, 50% loss).

4. We analyze the underlying failure mechanisms via bipedal fall rates and policy action sensitivity $\|\pi(\tilde{s}) - \pi(s)\|_2$.

2 Background and Related Work

2.1 Distributionally Robust and Adversarial MDPs

Classical Robust MDPs replace the fixed transition kernel P_0 with an uncertainty set $\mathcal{U}_\rho(P_0)$ and solve the max-min problem $\max_\pi \min_{P \in \mathcal{U}_\rho(P_0)} J(\pi, P)$. While traditional distributionally robust RL (DRRL) algorithms require a generative model to sample arbitrary state-action pairs, Liang et al. (2024) recently introduced **Distributionally Robust Q-learning (DRQ)**, the first model-free algorithm capable of learning distributionally robust policies from a *single online trajectory* via the dual formulation of f-divergence uncertainty sets and multi-timescale stochastic approximation.

2.2 Robustness Against Data Corruption and Observation Attacks

When an adversary corrupts training transitions with cumulative budget C, standard maximum likelihood estimation (MLE) in model-based RL degrades severely. Ye et al. (2024) proposed **CR-OMLE** and **CR-PMLE**, weighting transition samples by Total Variation (TV)-based information ratios to achieve an optimal additive regret bound $\tilde{\mathcal{O}}(\sqrt{T} + C)$.

On the observation side, Zhang et al. (2020) formalized the State-Adversarial MDP (SA-MDP), where an attacker perturbs the observed state $\tilde{s} = s + \delta$ within an ℓ_∞ ball while leaving the true physical state s unchanged. Recently, Zhou et al. (2025) demonstrated that adversarial training against a deterministic worst-case attacker causes policies to overfit to that specific attack direction; they proposed **ATSA**, a maximum-entropy stochastic adversary that generates diverse perturbations to improve generalization.

2.3 Unified Robust RL Benchmarks

Gu et al. (2025) introduced **Robust Gymnasium**, categorizing disruptions into four modular channels: observation, action, reward, and environment dynamics. Their benchmark evaluation revealed that neither standard RL algorithms (PPO, SAC) nor specialized robust methods maintain resilience across all disruption channels—motivating our systematic investigation into cross-channel fragility and robustness transfer.

3 Problem Formulation and Disturbance Taxonomy

Let $\mathcal{M}_{\text{train}} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ denote the nominal training MDP, and let $\pi_\theta : \mathcal{S} \to \mathcal{A}$ denote the deterministic evaluation policy (alongside twin critics $Q_{\phi_1}, Q_{\phi_2} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$) obtained after training on $\mathcal{M}_{\text{train}}$. At test time, the network parameters (θ, ϕ_1, ϕ_2) are frozen.

A test-time disturbance is parameterized by a pair (k, ϵ) representing the disturbance type k and intensity ϵ. As summarized in Table 1, the five disturbances enter the agent–environment loop at three distinct points:

Table 1: Taxonomy of the five test-time disturbances evaluated in this study.

Channel	Disturbance Type (k)	Mathematical Formulation	Intensity Unit (ϵ)
Sensor Input	Random Sensor Noise (`obs_noise`)	$\tilde{s}_t = s_t + \epsilon\sigma_s \odot \xi_t,\ \xi_t \sim \mathcal{N}(0, I)$	$\times$ per-dim state std σ_s
Sensor Input	Adversarial Attack (`obs_adv`)	$\tilde{s}_t = s_t + \arg\min_{\lvert\delta_i\rvert \le \epsilon\sigma_{s,i}} Q(s_t, \pi_\theta(s_t + \delta))$	$\times$ per-dim state std σ_s
Actuator	Action Delay (`act_delay`)	$\tilde{a}_t = a_{t-\epsilon}$ (FIFO queue of length ϵ)	Control steps ($\mathbb{Z}_{\ge 0}$)
Physics	Body Mass Scale (`mass`)	$m_b \leftarrow \epsilon m_b,\ I_b \leftarrow \epsilon I_b$ ($\forall$ bodies b)	Multiplier ($\epsilon = 1.0$ nominal)
Physics	Ground Friction Scale (`friction`)	$\mu_{\text{slide}} \leftarrow \epsilon\mu_{\text{slide}}$ ($\forall$ geoms)	Multiplier ($\epsilon = 1.0$ nominal)

Theoretical Distinction Between Channels. Sensor perturbations (`obs_noise` and `obs_adv`) do *not* alter the underlying physical transition kernel $P(s_{t+1} \mid s_t, a_t)$; rather, they corrupt the policy input $a_t = \pi_\theta(\tilde{s}_t)$. Conversely, actuation delay (`act_delay`) violates the first-order Markov assumption by coupling the transition to past actions $a_{t-\epsilon}$, while physical perturbations (`mass` and `friction`) directly modify the rigid-body dynamics $P_{\text{test}} \neq P_{\text{train}}$. Note that reward corruption is excluded from test-time evaluation because a frozen policy $\pi_\theta(s)$ does not observe rewards during deployment; reward poisoning strictly affects training-time Bellman updates.

White-Box Critic PGD Attack. Because state dimensions (joint angles in radians vs. angular velocities in rad/s) have heterogeneous scales, we first compute the empirical per-dimension standard deviation $\sigma_s \in \mathbb{R}^{17}$ across 5 clean trajectories of the trained policy. For the adversarial sensor attack (`obs_adv`) with budget ϵ, we solve the constrained minimization:

$$\delta_t^* = \arg\min_{\|\delta \oslash \sigma_s\|_\infty \le \epsilon} \min_{j \in \{1,2\}} Q_{\phi_j}\big(s_t,\ \pi_\theta(s_t + \delta)\big), \tag{1}$$

using $K = 10$ iterations of Projected Gradient Descent (PGD) initialized uniformly inside the ℓ_∞ box $[-\epsilon\sigma_s, +\epsilon\sigma_s]$ with step size $\alpha = 0.25\epsilon\sigma_s$:

$$\delta^{(m+1)} = \Pi_{[-\epsilon\sigma_s, \epsilon\sigma_s]}\left(\delta^{(m)} - \alpha \cdot \text{sign}\big(\nabla_\delta \min_{j=1,2} Q_{\phi_j}(s_t, \pi_\theta(s_t + \delta^{(m)}))\big)\right). \tag{2}$$

Critically, the critic $Q_{\phi_j}(s_t, \cdot)$ is evaluated at the *true* state s_t and the corrupted action $\pi_\theta(s_t + \delta)$, finding the sensor perturbation that forces the actor to choose the worst possible action for the current physical state (Zhang et al., 2020).

Evaluation Metrics. Let $J(\pi_\theta; k, \epsilon) = \mathbb{E}[\sum_{t=0}^{T} r_t]$ denote the mean empirical episode return of agent π_θ under disturbance (k, ϵ), and let $J_0 = J(\pi_\theta; \text{clean})$ denote its nominal clean return. We define:

- **Normalized Return:** $\bar{J}(k, \epsilon) = J(\pi_\theta; k, \epsilon)/J_0$, where $\bar{J} = 1.0$ indicates zero degradation.
- **Degradation Onset (ϵ_{80}) and Breaking Point (ϵ_{50}):** The smallest perturbation magnitude moving away from the nominal value at which $\bar{J}(k, \epsilon)$ drops below 0.80 (20% performance loss) and 0.50 (50% performance loss), respectively, computed via linear interpolation.
- **Fall Rate and Policy Sensitivity:** The fraction of episodes terminating early due to torso collapse (`Walker2d-v5`), and the mean ℓ_2 action shift $\|\pi_\theta(\tilde{s}_t) - \pi_\theta(s_t)\|_2$ induced by sensor perturbations.

4 Experimental Setup

Environments. We evaluate two MuJoCo locomotion environments from Gymnasium v1.x (Raffin et al., 2021), both featuring $\mathcal{S} \subset \mathbb{R}^{17}$ and $\mathcal{A} = [-1, 1]^6$:

1. `Walker2d-v5`: A planar bipedal robot with 6 actuated joints (left/right hip, knee, ankle). It requires active upright balance; an episode terminates early (a *fall*) if the torso height drops outside $[0.8, 2.0]$ m or torso pitch exceeds ± 1.0 rad. Control step $\Delta t = 8$ ms.
2. `HalfCheetah-v5`: A planar quadruped runner with 6 actuated joints. It is mechanically prevented from falling and always runs for the full horizon of $H = 1{,}000$ steps. Control step $\Delta t = 50$ ms.

SAC Training Configuration. We train Soft Actor-Critic (SAC) agents using Stable-Baselines3 across 3 independent random seeds $(0, 1, 2)$ per environment for 300,000 environment steps. Why SAC over tabular TD or Monte Carlo methods? Because $\mathcal{S} \subset \mathbb{R}^{17}$ and $\mathcal{A} = [-1, 1]^6$ are continuous, tabular storage is impossible and exact greedy maximization $\arg\max_{a \in \mathcal{A}} Q(s, a)$ cannot be enumerated. Moreover, Monte Carlo estimation suffers from high variance across 1,000-step horizons and cannot reuse off-policy data. SAC overcomes both limitations by combining

1-step off-policy TD learning on a replay buffer of size 10^6 (via twin MLP critics of size 256×256, $\gamma = 0.99$, Polyak $\tau = 0.005$, batch size 256, Adam learning rate 3×10^{-4}) with a continuous actor network $\pi_\theta(s)$ regularized by automatic maximum-entropy exploration.

Stress-Test Protocol. Each trained checkpoint is evaluated deterministically across 43 (disturbance, intensity) conditions for 10 episodes per condition (2,580 episodes total). Every condition uses identical environment reset seeds $\{0, 1, \ldots, 9\}$ (Common Random Numbers), ensuring that performance differences are attributable solely to the injected disturbance.

5 Results and Analysis

5.1 Clean Baseline Training Performance

Figure 1 displays the clean evaluation learning curves over 300,000 environment steps, and Table 2 summarizes the final baseline scores. Both environments converge smoothly with low inter-seed variance. `HalfCheetah-v5` achieves a clean return of 8245.5 ± 765.0, while `Walker2d-v5` achieves 3788.3 ± 287.6 with a low baseline fall rate of 6.7%.

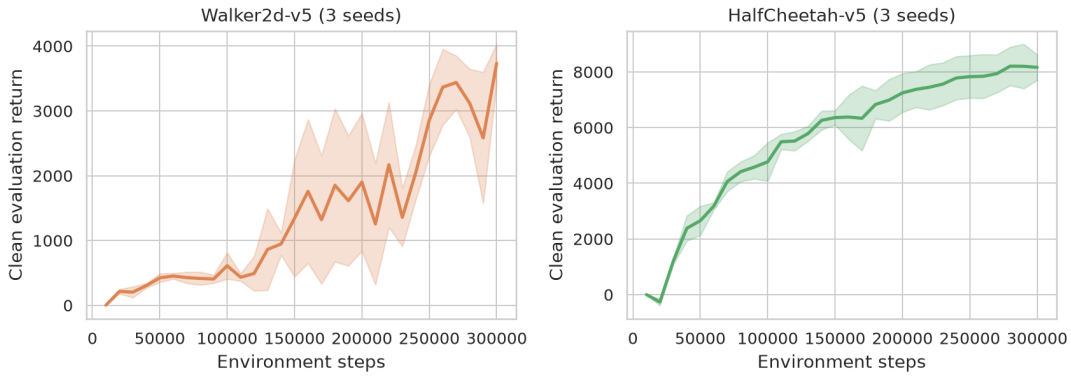


Figure 1: Clean evaluation return during SAC training across 300,000 environment steps (mean $\pm$ std across 3 seeds; evaluated every 10,000 steps over 5 episodes).

Table 2: Clean baseline performance of trained SAC agents after 300,000 steps (3 seeds $\times$ 10 evaluation episodes).

Environment	Clean Return (Mean $\pm$ Std)	Mean Episode Length	Clean Fall Rate
`HalfCheetah-v5`	8245.5 ± 765.0	1000.0	0.0% (cannot fall)
`Walker2d-v5`	3788.3 ± 287.6	980.2	6.7%

5.2 Degradation Curves and Breaking Points Across the 5 Disturbances

Figures 2 and 3 present the normalized and absolute return degradation curves across all five disturbances, while Table 3 reports the exact degradation onset (ϵ_{80}) and failure breaking points (ϵ_{50}). Figure 4 summarizes retained performance at pre-declared moderate disturbance levels.

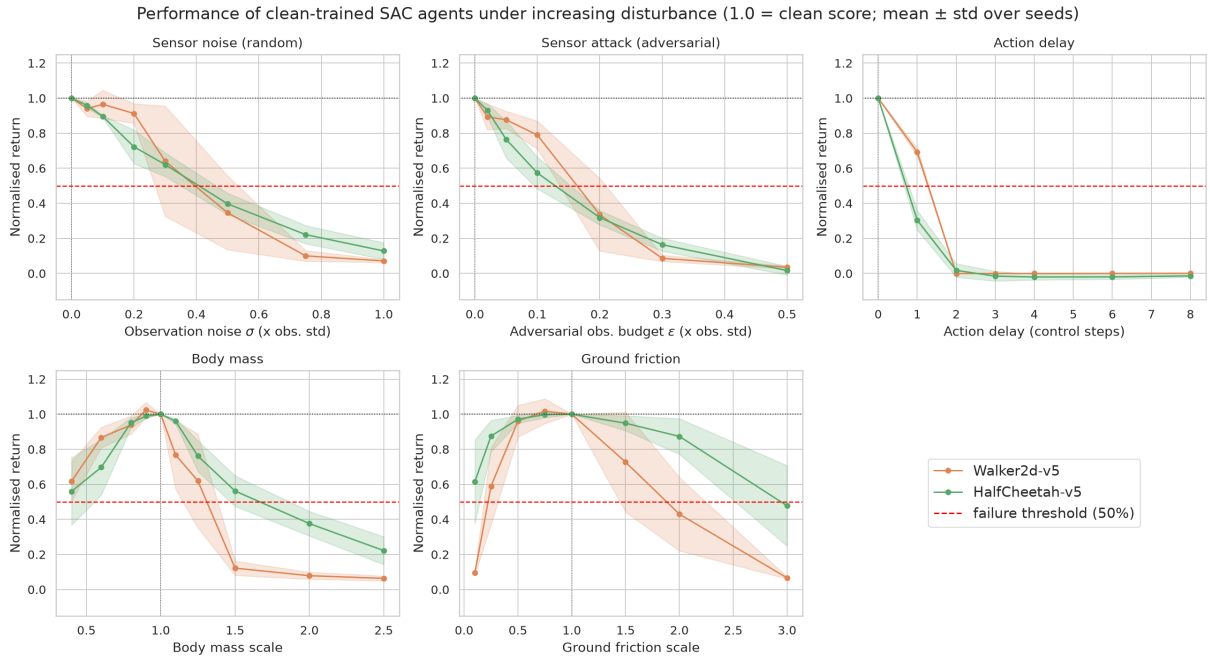


Figure 2: Normalized return ($\bar{J} = J_{\text{pert}}/J_{\text{clean}}$) of clean-trained SAC agents under increasing disturbance intensity across all five perturbation types (mean $\pm$ std over 3 seeds). The dashed red line marks the 50% failure threshold (ϵ_{50}).

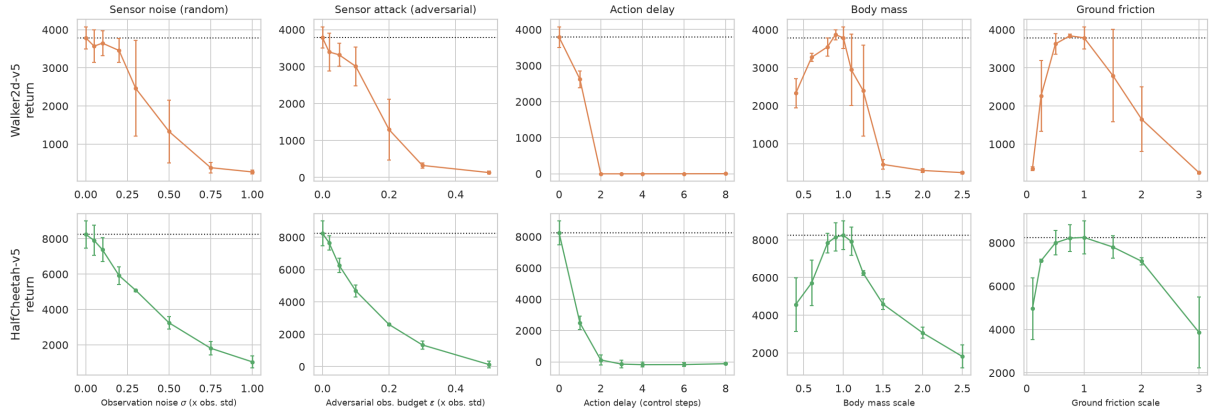


Figure 3: Absolute episode return versus disturbance intensity for `Walker2d-v5` (top row) and `HalfCheetah-v5` (bottom row). Dotted black lines indicate nominal clean baseline scores.

Table 3: Degradation onset (ϵ_{80}, where $\bar{J} < 0.80$), failure breaking point (ϵ_{50}, where $\bar{J} < 0.50$), and worst normalized return across all five disturbances ("∞" indicates threshold not crossed within tested range).

Disturbance Type	**Direction / Unit**	`Walker2d-v5` ($\epsilon_{80}/\epsilon_{50}$)	`HalfCheetah-v5` ($\epsilon_{80}/\epsilon_{50}$)	**Worst $\bar{J}$ (Walk / HC)**
Sensor Noise (`obs_noise`)	σ ($\times\sigma_s$)	0.241 / 0.395	0.155 / 0.408	0.071 / 0.128
Sensor Attack (`obs_adv`)	ϵ ($\times\sigma_s$)	0.094 / 0.164	0.044 / 0.129	0.035 / 0.016
Action Delay (`act_delay`)	Control steps	0.649 / 1.280	0.287 / 0.718	0.000 / $-$ 0.021
Body Mass (`mass`)	Heavier ($\uparrow$)	$\times$1.09 / $\times$ 1.31	$\times$1.22 / $\times$ 1.67	0.063 / 0.222
Body Mass (`mass`)	Lighter ($\downarrow$)	$\times$0.55 / ∞	$\times$0.68 / ∞	0.617 / 0.560
Ground Friction (`friction`)	Slippery ($\downarrow$)	$\times$0.39 / $\times$ 0.22	$\times$0.21 / ∞	0.094 / 0.616
Ground Friction (`friction`)	Sticky ($\uparrow$)	$\times$1.37 / $\times$ 1.88	$\times$2.19 / $\times$ 2.94	0.066 / 0.477

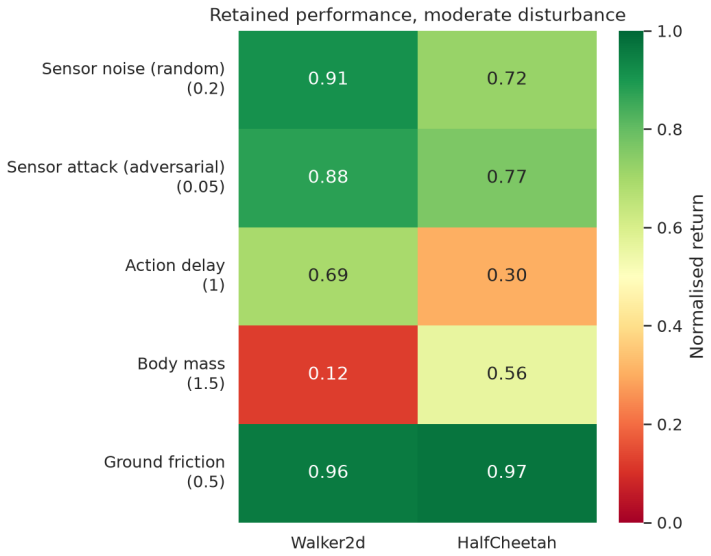


Figure 4: Retained normalized return under moderate disturbance intensities across both environments.

5.3 Random Sensor Noise vs. Adversarial Sensor Attack

Figure 5 directly compares random Gaussian sensor noise (`obs_noise`) against the 10-step PGD critic attack (`obs_adv`) on a shared logarithmic perturbation axis. Across both tasks, adversarial perturbations break the policy at $2.4\times$ **to** $3.2\times$ **smaller budgets**: on `Walker2d-v5`, ϵ_{50} drops from 0.395 to 0.164, while on `HalfCheetah-v5`, ϵ_{50} drops from 0.408 to 0.129. Notably, an adversarial sensor perturbation of just 4.4% of the standard deviation on `HalfCheetah-v5` reduces total return by 20%.

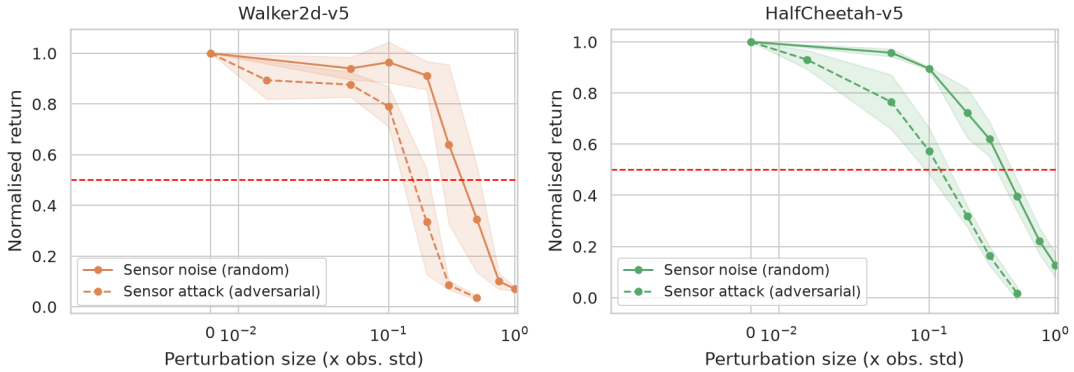


Figure 5: Comparison of random Gaussian sensor noise and white-box PGD adversarial sensor attacks at matched per-dimension budgets.

5.4 Failure Mechanisms: Bipedal Falls and Policy Sensitivity

Figure 6 illuminates *why* the policies fail:

1. **Abrupt Bipedal Collapse:** On `Walker2d-v5` (left and middle panels), performance degradation is driven by sudden torso falls. Under adversarial sensor attacks at $\epsilon = 0.2$, the fall rate jumps from 6.7% to 90%; under a 2-step action delay, the fall rate reaches 100%.
2. **Directional Sensitivity of Neural Policies:** The right panel plots the mean action deviation $\|\pi_\theta(\tilde{s}) - \pi_\theta(s)\|_2$. Under random Gaussian noise, action deviation grows linearly with slope ≈ 1.9 (reflecting the average local

Lipschitz constant of the actor MLP). Under PGD attack, the action deviation is **nearly twice as large** at every budget ϵ, confirming that PGD discovers steep gradient directions of the policy and critic networks.

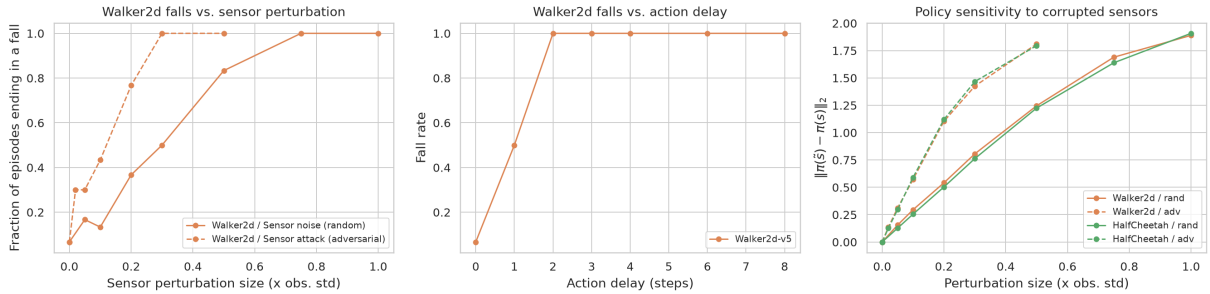


Figure 6: Left: `Walker2d-v5` fall rate under random vs. adversarial sensor perturbations. Middle: `Walker2d-v5` fall rate vs. action delay. Right: Policy action sensitivity $\|\pi_\theta(\tilde{s}) - \pi_\theta(s)\|_2$.

6 Discussion of Key Findings

1. Why is Action Delay the Most Destructive Disturbance? A single control step of latency reduces return by 31% on `Walker2d-v5` and 70% on `HalfCheetah-v5`, and 2 steps completely destroys both policies ($\bar{J} \leq 0$). Two factors explain this: first, standard MLP policies $\pi_\theta(s_t)$ are memoryless and assume the environment is strictly Markovian; when a_{t-k} is executed instead of a_t, the true system state includes the queue of k pending actions, rendering the MDP partially observable. Second, `HalfCheetah-v5` uses a control timestep of $\Delta t = 50$ ms (frame-skip 5×10 ms), whereas `Walker2d-v5` uses $\Delta t = 8$ ms. Thus, a 1-step delay corresponds to 50 ms of physical lag in `HalfCheetah-v5`—over $6\times$ longer in wall-clock physics time than in `Walker2d-v5`.

2. Why Are Physical Mass Shifts Asymmetric? Both agents tolerate lighter body masses well (retaining $> 56\%$ return even at $0.4\times$ mass, never crossing ϵ_{50}), but fail rapidly when mass increases ($\epsilon_{50} = 1.31\times$ for `Walker2d-v5` and $1.67\times$ for `HalfCheetah-v5`). Because SAC policies maximize forward velocity, they operate near actuator torque saturation limits $[-1, 1]^6$. Increasing body mass increases gravitational load $g(q)$ and inertial resistance $M(q)\ddot{q}$ beyond the maximum torque authority of the motors.

3. Robot Mechanics Dictate Which Physics Shift Hurts Most. On slippery surfaces ($0.22\times$ friction), `Walker2d-v5` loses 50% of its return because foot slippage immediately causes its center of mass to exit its narrow bipedal support polygon, resulting in a fall. In contrast, `HalfCheetah-v5` cannot fall over and retains 61.6% of its clean score even on near-frictionless ice ($0.1\times$ friction). Consequently, there is no universal "most damaging" physical parameter across tasks.

7 Limitations and End-Semester Roadmap

Limitations of Current Study. First, our evaluation applies disturbances strictly at *test time* across 3 seeds per environment; training-time reward corruption requires retraining agents under poisoned reward signals and will be conducted in the second phase. Second, because perturbation units differ across channels (σ_s multiples vs. delay steps vs. mass multipliers), raw breaking points cannot be compared across channels without magnitude calibration.

Plan for End-Semester Phase. In the second half of the semester, we will transition from *diagnosis* to *defense and transfer*:

1. **Implement Robust Training Strategies:** Observation-noise augmentation, policy smoothness/Lipschitz regularization (Zhang et al., 2020), domain randomization over mass and friction, distributionally robust pessimistic Q-backups (Liang et al., 2024), and uncertainty-weighted TD losses against reward corruption (Ye et al., 2024).
2. **Cross-Robustness Transfer Matrix:** Systematically train an agent using a defense designed for channel k and evaluate it under channel $k' \neq k$ at performance-calibrated intensities to test whether robustness transfers across perturbation families.

3. **Unified Multi-Channel Stochastic Adversary:** Inspired by ATSA (Zhou et al., 2025), we will design a maximum-entropy adversary that stochastically perturbs observations, actions, and dynamics during training, accompanied by ablation studies.

References

Gu, S., Shi, L., Wen, M., Jin, M., Mazumdar, E., Chi, Y., Wierman, A., and Spanos, C. (2025). Robust gymnasium: A unified modular benchmark for robust reinforcement learning. In *International Conference on Learning Representations (ICLR)*.

Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, pages 1861–1870. PMLR.

Liang, Z., Ma, X., Blanchet, J., Yang, J., Zhang, J., and Zhou, Z. (2024). Single-trajectory distributionally robust reinforcement learning. In *International Conference on Machine Learning (ICML)*, pages 29591–29613. PMLR.

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., and Dormann, N. (2021). Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8.

Ye, C., He, J., Gu, Q., and Zhang, T. (2024). Towards robust model-based reinforcement learning against adversarial corruption. In *International Conference on Machine Learning (ICML)*. PMLR.

Zhang, H., Chen, H., Xiao, C., Li, B., Liu, M., Boning, D., and Hsieh, C.-J. (2020). Robust deep reinforcement learning against adversarial perturbations on state observations. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 21024–21037.

Zhou, Z., Liu, G., Zhou, M., and Guo, W. (2025). Robust multi-agent reinforcement learning with stochastic adversary. In *International Conference on Machine Learning (ICML)*. PMLR.